

Lokalizacija zvuka mikrofonskim nizom

STM32G474 (akvizicija, DSP i IMU) + ESP32 (Wi-Fi 3D vizualizacija)

Ovo izdanje proširuje drugu verziju s dva nova poglavlja koja pokrivaju dvije značajne nadogradnje sustava — **senzor orijentacije (IMU)** i **lokalizaciju u vremenskoj domeni**:

- **IMU senzor (MPU-6500/9250)** — kako STM32 preko **I2C** sabirnice čita akcelerometar i žiroskop, kako se **WHO_AM_I** registrom prepoznaje točan čip, kako **Mahony** fuzija pretvara sirova mjerenja u kutove nagiba (pitch/roll/yaw), te kako se orijentacija šalje na ESP32 (paket 0x05) i koristi za **zakretanje 3D scene** u pregledniku.
- **Lokalizacija u vremenskoj domeni** — alternativa GCC-PHAT-u koja radi **bez FFT-a**: traži **trenutak dolaska** (onset) zvuka po kanalu pragom na amplitudi, iz razlike onseta računa TDOA. Objašnjavamo geometriju **pravilnog tetraedra**, razliku između verzije s 3 mikrofona (uz pretpostavku $z \geq 0$) i 4 mikrofona (puni 3D sustav), te kada koji pristup koristiti.

Ostatak dokumenta (arhitektura, Fourier/FFT, FreeRTOS, akvizicija, GCC-PHAT lokalizacija, UART protokol, ESP32 vizualizacija, cjeloviti primjer i pojmovnik) zadržan je i dopunjen iz prethodnih verzija.

Prethodne verzije sačuvane su kao «Dokumentacija_Lokalizacija_Zvuka_v1.pdf» i «..._v2.pdf».

Sadržaj

1. Pregled sustava — što i zašto
2. Arhitektura: dvije ploče, jedan tok podataka
3. Hardver i geometrija mikrofonskog niza
4. Matematički temelji: Fourierova transformacija i FFT
5. FreeRTOS od nule: taskovi, stog, prioriteti, queue/semafor
6. STM32 lanac akvizicije: TIM8 → ADC → DMA → ISR
7. FreeRTOS u ovom projektu: taskovi i protok podataka
8. Procjena veličine stoga u ovom projektu
9. Detekcija i lokalizacija zvuka — korak po korak (GCC-PHAT)
10. Lokalizacija u vremenskoj domeni (3 i 4 mikrofona)
11. IMU senzor (MPU-6500/9250): I2C i orijentacija plohe
12. UART protokol između STM32 i ESP32
13. ESP32: Wi-Fi, web-poslužitelj i 3D vizualizacija
14. Cjeloviti primjer: od pljeska do točke na ekranu
15. Pojmovnik za početnike

1. Pregled sustava — što i zašto

Cilj sustava je **odrediti smjer iz kojeg dolazi zvuk** (npr. pljesak rukama) i taj smjer prikazati u stvarnom vremenu kao točku u 3D prostoru u web-pregledniku. Sustav ne prepoznaje *što* je zvuk, nego *odakle* dolazi — njegov azimut (kut u vodoravnoj ravnini) i elevaciju (kut prema gore).

Princip rada temelji se na jednostavnoj fizikalnoj činjenici: zvuk putuje konačnom brzinom (oko 343 m/s u zraku). Ako su mikrofoni razmaknuti u prostoru, isti zvučni val do njih stiže u **malo različitim trenucima**. Ta razlika u vremenu dolaska zove se **TDOA** (Time Difference Of Arrival). Iz nekoliko TDOA mjerenja i poznatih položaja mikrofona moguće je geometrijski izračunati smjer izvora.

Intuicija u jednoj rečenici

Ako mikrofoni M2 «čuje» pljesak 0.2 ms prije mikrofona M1, izvor je bliži mikrofoni M2 — a koliko točno «u stranu», govori nam omjer kašnjenja na svim parovima mikrofona.

Sustav je podijeljen na dvije ploče s jasno odvojenim ulogama:

- **STM32G474** — «mozak za obradu signala». Uzorkuje 4 mikrofona, obavlja svu matematiku (FFT, korelacija, geometrija) i kao rezultat šalje samo tri broja: azimut, elevaciju i jakost.
- **ESP32** — «prozor prema korisniku». Prima te brojeve preko UART-a, podiže vlastitu Wi-Fi mrežu i poslužuje web-stranicu s 3D prikazom. Ne radi nikakvu obradu zvuka.

Ovakva podjela tipičan je obrazac u ugradbenim sustavima: **mikrokontroler specijaliziran za determinističku obradu u stvarnom vremenu** (STM32 s FPU-om i DSP knjižnicom) odvojen je od **mikrokontrolera specijaliziranog za povezivost** (ESP32 s Wi-Fi/TCP-IP stogom). Svaki radi ono u čemu je dobar, a povezuje ih jednostavan serijski kabel.

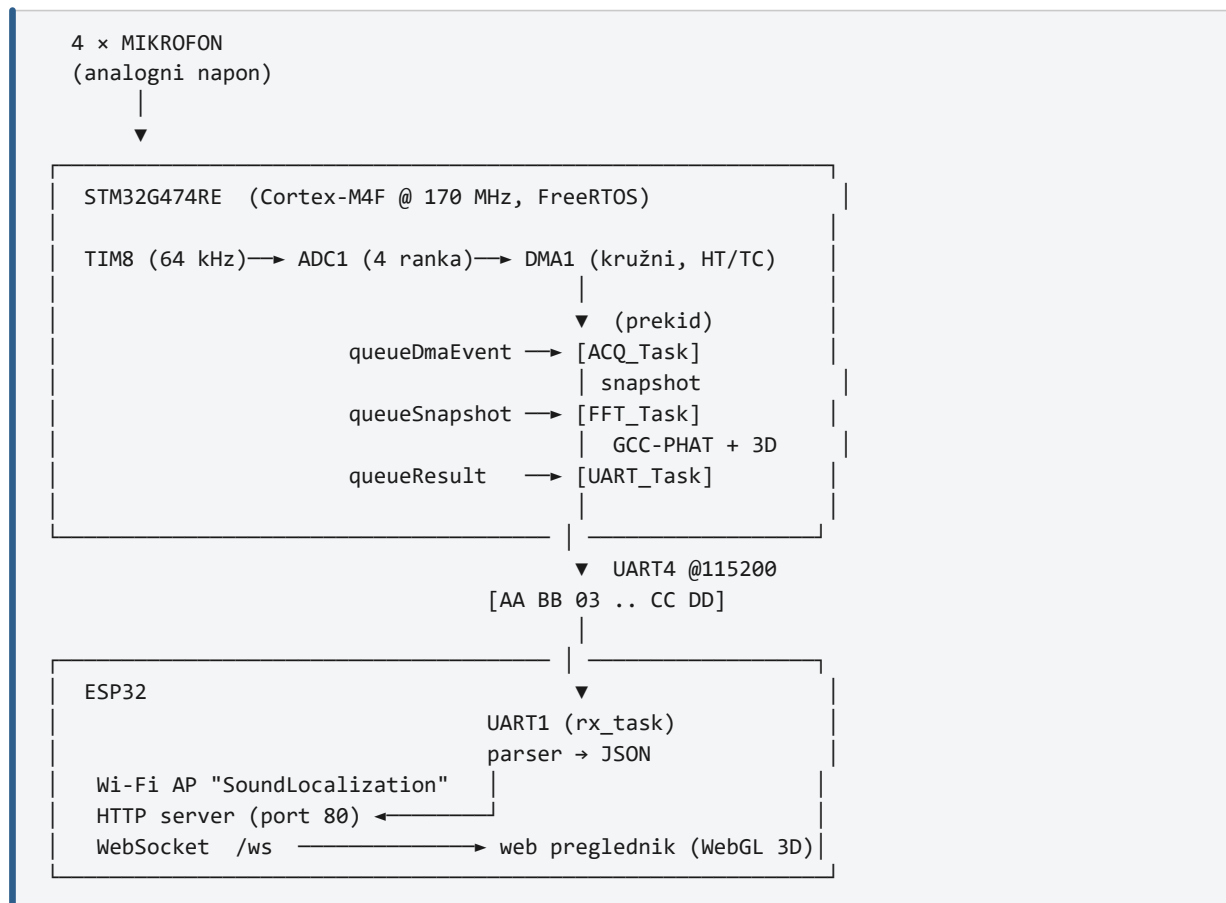
2. Arhitektura: dvije ploče, jedan tok podataka

Cijeli sustav najlakše je razumjeti kao **cjevovod** (pipeline): podatak ulazi kao zvučni tlak na mikrofoni, a izlazi kao piksel na ekranu. Na svakom koraku podatak mijenja oblik i postaje «sažetiji».

Korak	Oblik podatka	Količina
Zvučni val	tlak zraka	—
Mikrofon + pretpojačalo	analogni napon 0–3.3 V	4 kanala
ADC (TIM8 okida)	12-bitni uzorci (0–4095)	256 kS/s ukupno
DMA u RAM	interleaveani uint16 buffer	8192 uzoraka
DSP (FFT/GCC-PHAT)	kutovi azimut/elevacija	≈ 5 brojeva
UART paket	10 bajtova (tip 0x03)	po detekciji
ESP32 → WebSocket	JSON tekst	po detekciji
Web preglednik	3D točka (WebGL)	vizualno

Tablica 2. Tok podataka — svaki korak smanjuje količinu, a povećava «značenje» podatka.

2.1 Blok-shema



Slika 1. Blok-shema cijelog sustava: fizički signal → digitalna obrada → mreža → ekran.

Primijetite tri **reda čekanja** (queue) unutar STM32: oni razdvajaju dijelove sustava koji rade različitom brzinom. Prekidna rutina (ISR) i taskovi nikada ne dijele podatke izravno — uvijek preko reda čekanja. Zašto je to važno objašnjava poglavlje 5; kako je iskorišteno, poglavlje 7.

3. Hardver i geometrija mikrofonskog niza

Točnost lokalizacije izravno ovisi o tome koliko **precizno poznajemo položaje mikrofona**. Cijela matematika polazi od koordinata zadanih u datoteci `audio_common.h`. Mikrofon M1 je u ishodištu i služi kao **referentni** — svi se TDOA mjere u odnosu na njega.

Mikrofon	X [cm]	Y [cm]	Z [cm]	ADC rank	Pin
M1 (ref.)	0.00	0.00	0.00	RANK1	PB14 / IN5
M2	8.67	5.00	0.00	RANK2	PC0 / IN6
M3	8.67	-5.00	0.00	RANK3	PC1 / IN7
M4 (vrh)	5.00	0.00	8.00	RANK4	PC2 / IN8

Tablica 3. Položaji mikrofona i ADC kanali. M1–M2–M3 čine ~jednakostranični trokut u ravnini, M4 je iznad baze.

Raspored je odabran promišljeno: tri mikrofona (M1, M2, M3) leže u vodoravnoj ravnini i daju **azimut**, dok je četvrti (M4) podignut u visinu ($Z = 8$ cm) i tek on omogućuje mjerenje **elevacije**. Bez mikrofona izvan ravnine sustav ne bi mogao razlikovati zvuk odozgo od zvuka odozdo.

3.1 Koordinatni sustav i značenje kutova

- **+X** → 0° azimut: «naprijed» (simetrala između M2 i M3).
- **+Y** → $+90^\circ$ azimut: lijevo (strana mikrofona M2).

- **-Y** → -90° azimut: desno (strana mikrofona M3).
- **-X** → ±180° azimut: nazad (strana mikrofona M1).
- **+Z** → +90° elevacija: ravno gore (smjer mikrofona M4).

Zašto baš ~10 cm razmaka

Veći razmak → veće (lakše mjerljivo) kašnjenje → bolja kutna rezolucija. Ali ako je razmak veći od pola valne duljine najviših frekvencija, javlja se prostorni «aliasing» (dvosmislenost smjera). Razmak ~10 cm pri 343 m/s daje najveće kašnjenje od oko 18.7 uzoraka na 64 kHz — odatle konstanta `TDOA_MAX_SAMPLES = 20`.

Računica granice: najveći razmak od referentnog mikrofona je ~10 cm. Vrijeme da zvuk prijeđe 10 cm je $0.10 \text{ m} \div 343 \text{ m/s} = 291.5 \text{ } \mu\text{s}$. Pri 64 000 uzoraka/s to je $291.5 \text{ } \mu\text{s} \times 64 \text{ 000} = \mathbf{18.66 \text{ uzoraka}}$. Zaokruženo naviše s rezervom → 20.

4. Matematički temelji: Fourierova transformacija i FFT

Ovo poglavlje gradi razumijevanje FFT-a od nule, na malim primjerima koje možete pratiti olovkom na papiru. Tek u zadnjem odjeljku (4.9) povezujemo to s konkretnom implementacijom u ovom projektu. Ako vam je gradivo poznato, možete preskočiti na 4.9.

4.1 Dvije domene: vrijeme i frekvencija

Signal s mikrofona prirodno «vidimo» u **vremenskoj domeni**: niz vrijednosti amplitude u jednakim vremenskim razmacima (uzorci). To odgovara na pitanje «kolika je amplituda u trenutku t?».

Fourierova transformacija isti signal prikazuje u **frekvencijskoj domeni**: kao recept koji kaže **od kojih je sinusoida (i kojih jakosti i faza) signal sastavljen**. To odgovara na pitanje «koliko ima koje frekvencije u signalu?». Oba prikaza nose istu informaciju — samo posloženu drugačije.

Analogija: glazbeni akord

Vremenska domena je titranje zraka koje uho prima. Frekvencijska domena je popis nota u akordu (npr. C, E, G) i koliko je svaka glasna. Fourier je «uho koje raščlani akord na pojedine note».

4.2 Ideja: svaki signal je zbroj sinusoida

Temeljni Fourierov uvid: gotovo svaki signal može se napisati kao zbroj sinusa i kosinusa različitih frekvencija. Niska frekvencija opisuje spore promjene, visoka frekvencija oštre/brze promjene. «Težine» u tom zbroju (koliko svake frekvencije ima) upravo su ono što transformacija računa.

Budući da računalo radi s konačnim nizom uzoraka (a ne s neprekidnom funkcijom), koristimo **diskretnu** Fourierovu transformaciju — DFT.

4.3 DFT — formula i značenje svakog simbola

Za niz od N uzoraka $x[0], x[1], \dots, x[N-1]$, DFT daje N kompleksnih brojeva $X[0], \dots, X[N-1]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{(-j \cdot 2\pi \cdot k \cdot n / N)} \quad k = 0, 1, \dots, N-1$$

Formula DFT-a. $X[k]$ mjeri «koliko ima» frekvencije koja u prozor stane točno k puta.

- **$x[n]$** — n -ti uzorak u vremenu (realan broj kod nas).
- **$X[k]$** — k -ti «bin» (frekvencijski pretinac), kompleksan broj.

- **k** — koliko punih titraja ta frekvencija napravi unutar prozora od N uzoraka. $k=0$ je istosmjerna (DC) komponenta, $k=1$ je jedan titraj po prozoru, itd.
- $e^{(-j \cdot 2\pi \cdot k \cdot n / N)}$ — jedinična «sonda»: rotirajući kompleksni broj (fazor) frekvencije k . Pritom je j imaginarna jedinica ($j^2 = -1$).
- Σ — zbroj po svim uzorcima: množimo signal sa sondom i zbrajamo.

Ključ je **Eulerova formula**, koja kompleksni eksponent pretvara u sinus i kosinus:

$$e^{(-j\theta)} = \cos(\theta) - j \cdot \sin(\theta)$$

$$\Rightarrow X[k] = \sum x[n] \cdot \cos(2\pi \cdot k \cdot n / N) - j \cdot \sum x[n] \cdot \sin(2\pi \cdot k \cdot n / N)$$

______ realni dio ______ / ______ imaginarni dio ______ /

DFT preko sinusa/kosinusa: realni dio «koliko liči na kosinus», imaginarni «koliko na sinus» te frekvencije.

Intuitivno: za svaki k uspoređujemo signal s kosinusom i sinusom te frekvencije. Ako se dobro poklapaju, zbroj je velik $\rightarrow$ te frekvencije ima puno. Ako se ne poklapaju, doprinosi se međusobno poništavaju $\rightarrow$ zbroj je ~ 0 .

4.4 Primjer 1 — DFT na 4 uzorka, ručni izračun

Uzmimo najjednostavniji slučaj $N = 4$ i signal $x = [1, 2, 3, 4]$. Za $N = 4$ sonda se silno pojednostavi jer je $e^{(-j \cdot 2\pi / 4)} = e^{(-j\pi/2)} = -j$, pa je $e^{(-j \cdot 2\pi \cdot k \cdot n / 4)} = (-j)^{(k \cdot n)}$. Potencije od $(-j)$ kruže:

$$(-j)^0 = 1 \quad (-j)^1 = -j \quad (-j)^2 = -1 \quad (-j)^3 = +j \quad (\text{pa se ponavlja})$$

Sada uvrstimo za svaki k ($k \cdot n$ po modulu 4):

$$\begin{aligned} X[0] &= 1 \cdot 1 + 2 \cdot 1 + 3 \cdot 1 + 4 \cdot 1 = 1+2+3+4 = 10 \\ X[1] &= 1 \cdot 1 + 2 \cdot (-j) + 3 \cdot (-1) + 4 \cdot (+j) = (1-3) + (-2+4)j = -2 + 2j \\ X[2] &= 1 \cdot 1 + 2 \cdot (-1) + 3 \cdot 1 + 4 \cdot (-1) = 1-2+3-4 = -2 \\ X[3] &= 1 \cdot 1 + 2 \cdot (+j) + 3 \cdot (-1) + 4 \cdot (-j) = (1-3) + (2-4)j = -2 - 2j \end{aligned}$$

Rezultat: $X = [10, -2+2j, -2, -2-2j]$. Primijetite $X[3] = \text{konjugat od } X[1]$ — tipično za realan ulaz.

Provjera DC-a: $X[0] = 10$ je samo zbroj svih uzoraka, tj. N puta prosjek (prosjek = 2.5). To uvijek vrijedi — **bin 0 je istosmjerna komponenta**.

Zašto je $X[3]$ zrcalna kopija $X[1]$

Kad je ulaz realan, druga polovica spektra je «konjugirano zrcalo» prve ($X[N-k] = \text{konjugat } X[k]$). Zato u praksi za realan signal čuvamo samo prvih $N/2+1$ binova — ostatak ne nosi novu informaciju. Tu činjenicu koristi i «realni FFT» (4.9).

4.5 Primjer 2 — čisti ton postaje šiljak u spektru

Uzmimo $N = 8$ i signal koji je **točno jedan** puni sinus po prozoru, $x[n] = \sin(2\pi \cdot 1 \cdot n / 8)$ (dakle frekvencija bina $k = 1$):

n	:	0	1	2	3	4	5	6	7
x[n]	:	0.00	0.71	1.00	0.71	0.00	-0.71	-1.00	-0.71

DFT takvog čistog sinusa daje gotovo sve nule, osim dva bina:

X[k]	:	0	4	0	0	0	0	0	4
k	:	0	(1)	2	3	4	5	6	(7)

Sva «energija» je u binu $k=1$ (i njegovom zrcalu $k=7$). Drugim riječima: jedna frekvencija $\rightarrow$ jedan šiljak.

Ovo je suština: **spektar pokazuje koje frekvencije postoje**. Sinus jedne frekvencije pokaže se kao oštar vrh točno na svom binu; složeniji zvuk (npr. pljesak) raspodijeli energiju po mnogo binova. Iznos $4 = N/2$ dolazi od konvencije normalizacije DFT-a (amplituda $1 \rightarrow N/2$ po svakoj od dvije zrcalne polovice).

4.6 Magnituda i faza — što nam $X[k]$ govori

Svaki $X[k]$ je kompleksan broj $X[k] = a + j \cdot b$ i ima dvije korisne veličine:

- **Magnituda** $|X[k]| = \sqrt{a^2 + b^2}$: *koliko* ima te frekvencije (jakost/amplituda).
- **Faza** $X[k] = \text{atan2}(b, a)$: *s kojim pomakom* (kašnjenjem) ta frekvencija počinje.

Zašto je faza ključna za ovaj projekt

Kašnjenje signala u vremenu (TDOA!) u frekvencijskoj se domeni očituje kao **linearna promjena faze** s frekvencijom. Upravo zato metoda GCC-PHAT (poglavlje 9) radi s fazom: kašnjenje između dva mikrofona «pročita» iz razlike faza njihovih spektara. Magnitudu čak namjerno odbacuje.

4.7 Zašto FFT: ista matematika, drastično manje računa

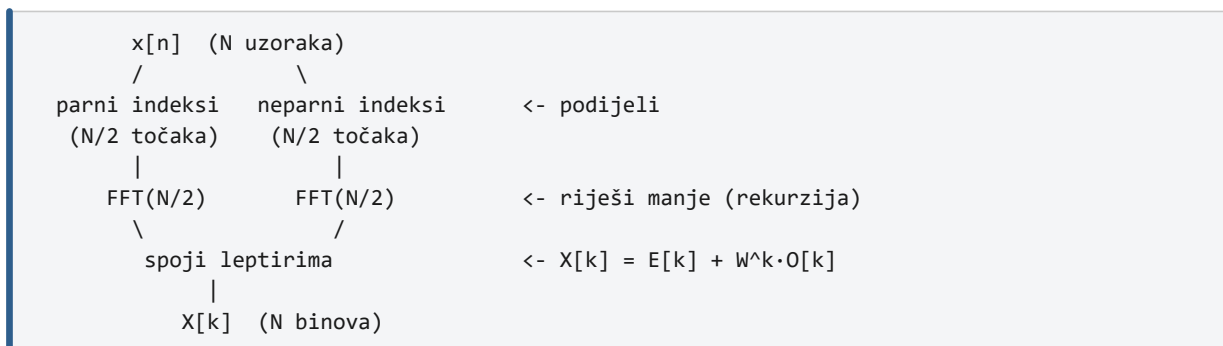
Izravan izračun DFT-a po formuli traži, za svaki od N binova, zbroj od N članova $\rightarrow N^2$ operacija. Za $N = 1024$ to je preko milijun množenja po transformaciji — pretjerano za stvarno vrijeme.

FFT (Fast Fourier Transform) je *algoritam* koji računa **isti rezultat**, ali pametno iskorištava simetrije pa mu treba samo $N \cdot \log_2 N$ operacija:

N (veličina)	DFT izravno (N^2)	FFT ($N \cdot \log_2 N$)	Ubrzanje
8	64	24	$\approx 2.7\times$
64	4 096	384	$\approx 11\times$
1024	1 048 576	10 240	$\approx 102\times$
4096	16 777 216	49 152	$\approx 341\times$

Tablica 4. FFT je to isplativiji što je N veći. Zahtijeva da je N potencija broja 2.

Trik FFT-a (Cooley–Tukey): rastavi N -točkovni problem na dva $N/2$ -točkovna (uzorci na parnim i na neparnim mjestima), riješi njih, pa rezultate spoji. To se rekurzivno ponavlja. Osnovna operacija spajanja zove se **leptir** (butterfly).



Slika 2. «Podijeli pa vladaj»: $\log_2 N$ razina, svaka $N/2$ leptira $\rightarrow N \cdot \log_2 N$ operacija.

4.8 Primjer 3 — FFT na 4 uzorka preko leptira

Provjerimo da FFT daje isti rezultat kao izravni DFT iz Primjera 1, $x = [1, 2, 3, 4]$. Prvo razdvojimo na parne i neparne uzorke:

```

parni    = [x0, x2] = [1, 3]
neparni  = [x1, x3] = [2, 4]

2-točkovni DFT (samo zbroj/razlika):
E = DFT[1,3] = [1+3, 1-3] = [ 4, -2]      (E[0], E[1])
O = DFT[2,4] = [2+4, 2-4] = [ 6, -2]      (O[0], O[1])

```

Korak 1: dva mala 2-točkovna DFT-a. Za $N=2$ DFT je samo $(a+b)$ i $(a-b)$.

Sada spajamo leptirom, uz «twiddle» faktor $W = e^{-j \cdot 2\pi/4} = -j$, koristeći $X[k] = E[k] + W^k \cdot O[k]$ za prvu polovicu i $X[k+N/2] = E[k] - W^k \cdot O[k]$ za drugu:

```

X[0] = E[0] + W^0 \cdot O[0] = 4 + (1)(6) = 10
X[1] = E[1] + W^1 \cdot O[1] = -2 + (-j)(-2) = -2 + 2j
X[2] = E[0] - W^0 \cdot O[0] = 4 - (1)(6) = -2
X[3] = E[1] - W^1 \cdot O[1] = -2 - (-j)(-2) = -2 - 2j

```

Korak 2: leptiri. Rezultat $X = [10, -2+2j, -2, -2-2j]$ — identičan Primjeru 1, uz manje računa.

Što smo upravo vidjeli

Isti odgovor, ali umjesto $4^2 = 16$ «punih» množenja, FFT je trebao tek nekoliko zbrajanja i jedno netrivialno množenje s twiddle faktorom. Na $N = 1024$ ta razlika znači obradu u mikrosekundama umjesto milisekundama.

4.9 Realni FFT i pakiranje rezultata (priprema za projekt)

Naš ulaz (uzorci s mikrofona) je **realan**, pa je druga polovica spektra zrcalna (vidi 4.4). «Realni FFT» to iskorištava i radi gotovo dvostruko brže od kompleksnog te vraća samo neredundantne binove ($0 \dots N/2$). CMSIS-DSP funkcija `arm_rfft_fast_f32` to pakira ovako:

```

fft[0]      = bin 0 (DC)          - realan
fft[1]      = bin N/2 (Nyquist)   - realan, spremljen u imaginarni slot DC-a
fft[2k]     = realni dio bina k    (k = 1 .. N/2-1)
fft[2k+1]   = imaginarni dio bina k

```

Pakiranje izlaza realnog FFT-a u CMSIS-DSP. Štedi memoriju: N realnih ulaza $\rightarrow N$ realnih izlaza.

4.10 Hann prozor i «curenje» spektra

DFT pretpostavlja da je prozor uzoraka jedan period beskonačno ponavljajućeg signala. Ako signal na kraju prozora ne «sjedne» glatko na početak, nastaje oštar skok koji u spektru stvori lažne frekvencije — **spektralno curenje** (leakage). Lijek je **prozorska funkcija** koja rubove glatko spušta na nulu.

```

Hann prozor:  w[n] = 0.5 \cdot (1 - cos(2\pi \cdot n / (N-1))),  n = 0 .. N-1

signal[n]  —x—  w[n]  —>  «omekšani» signal za FFT
                    (zvono: 0 na rubovima, 1 u sredini)

```

Hann prozor množi signal glatkim zvonom prije FFT-a i tako smanjuje curenje.

4.11 FFT u ovom projektu

Sada možemo precizno opisati kako je FFT iskorišten. Pri pokretanju se jednom pripreme FFT instanca i tablica Hann prozora (`GCC_Init`), a zatim se za svaki par mikrofona rade dvije unaprijedne i jedna inverzna transformacija.

Parametar	Vrijednost	Značenje
FFT_SIZE (N)	1024	uzoraka po kanalu u jednom prozoru
Brzina uzorkovanja	64 000 Hz	—
Trajanje prozora	$1024 / 64000 = 16 \text{ ms}$	vremenska duljina analize
Rezolucija bina	$64000 / 1024 = 62.5 \text{ Hz}$	razmak između susjednih frekvencija
Nyquistova granica	32 000 Hz	najviša prikaziva frekvencija ($N/2$ bin)
Knjižnica	CMSIS-DSP	arm_rfft_fast_f32 (realni FFT, hardverski FPU)

Tablica 5. Parametri FFT-a u projektu (audio_common.h, gcc_phat.c).

```

void GCC_Init(void) {
    for (int i = 0; i < FFT_SIZE; i++)
        s_hann[i] = 0.5f*(1.0f - cosf(2.0f*PI*i/(FFT_SIZE-1))); /* Hann */
    arm_rfft_fast_init_f32(&s_rfft, FFT_SIZE); /* FFT setup */
}

/* po paru mikrofona (vidi poglavlje 9 – GCC-PHAT): */
arm_rfft_fast_f32(&s_rfft, ref, fft_x, 0); /* 0 = unaprijed (FFT) */
arm_rfft_fast_f32(&s_rfft, sig, fft_y, 0);
... /* križni spektar + PHAT */
arm_rfft_fast_f32(&s_rfft, fft_c, corr, 1); /* 1 = inverzno (IFFT) */

```

Isječak 1. FFT priprema i poziv (gcc_phat.c). Detaljna uporaba u poglavlju 9.

Veličina 1024 je kompromis: dovoljno velik prozor da rezolucija (62.5 Hz) i vremenski raspon (16 ms) pokriju pljesak, a dovoljno malen da cijela obrada (2 FFT-a + 1 IFFT po paru, 3 para) stane u proračun vremena između dva DMA prekida (16 ms) bez gubljenja podataka.

5. FreeRTOS od nule: taskovi, stog, prioriteti, queue/semafor

Ovo poglavlje uvodi FreeRTOS na malom, samostalnom primjeru (mjerenje temperature i slanje). Na njemu učimo procijeniti stog, odabrati prioritet i izabrati ispravan mehanizam komunikacije. Kako je to primijenjeno u ovom projektu, slijedi u poglavljima 7 i 8.

5.1 Zašto RTOS, a ne obična «super-petlja»

Najjednostavniji firmware je jedna velika petlja (`while(1)`) koja redom radi sve poslove. To radi dok su poslovi kratki i jednako hitni. Problem nastaje kad jedan posao mora biti **brz i točan na vrijeme** (npr. preuzeti svjež blok zvuka), a drugi je **spor** (npr. slanje 8 KB preko UART-a koje traje 0.7 s).

U super-petlji bi spori posao «zaglavio» cijeli sustav i hitni bi posao zakasnio. **RTOS** (Real-Time Operating System) rješava to dijeleći program na **taskove** i automatski izmjenjujući ih na procesoru prema **prioritetu** — hitan task u svakom trenutku može «preuzeti» procesor od manje hitnog.

5.2 Task, raspoređivač i stanja

Task je neovisna funkcija koja se vrti u vlastitoj beskonačnoj petlji i ima **vlastiti stog**. **Raspoređivač** (scheduler) je dio RTOS-a koji odlučuje koji task trenutno koristi procesor. U svakom trenutku task je u jednom od stanja:

Stanje	Značenje
Running	Trenutno se izvršava na procesoru (samo jedan task na jednojezgrenom MCU-u).
Ready	Spreman je i čeka procesor; raspoređivač će ga pokrenuti čim bude najviši po prioritetu.
Blocked	Čeka na nešto (npr. podatak iz reda, istek vTaskDelay) i NE troši procesor.
Suspended	Ručno zaustavljen; ne sudjeluje u raspoređivanju dok se ne nastavi.

Tablica 6. Stanja taska. Ključ za uštedu energije/CPU-a: task koji čeka treba biti Blocked, ne u praznoj petlji.

Najvažnije pravilo za početnike

Task NIKAD ne smije «vrtjeti prazno» čekajući (busy-wait). Umjesto toga blokira na redu čekanja, semaforu ili vTaskDelay — tada troši 0% procesora i pušta druge taskove da rade. Blokiranje je dobro, ne loše.

5.3 Prioriteti i preemptivni raspored

Svaki task dobiva **prioritet** (veći broj = viši prioritet). Pravilo preemptivnog raspoređivača: **uvijek se izvršava task najvišeg prioriteta koji je Ready**. Ako task višeg prioriteta postane spreman (npr. stigne mu podatak), on odmah «preuzima» procesor od nižeg.

Primjer: tri taska, brojevi = prioritet (3 najviši)

vrijeme →

A(3) hitan : ■ ■ ■

B(2) srednji : ■■■ ■■■ ■■■

C(1) spori : ■■■ ■■■ ■■■ ■■■ ■■■

(radi samo kad su A i B Blocked)

■ = task drži procesor. Kad A postane Ready, prekida B ili C (preemption).

Slika 3. Viši prioritet uvijek pobjeđuje. Niski task radi tek kad viši «spavaju» (Blocked).

5.4 Mali primjer: senzor temperature + slanje

Napravimo minimalan sustav s dva taska. **Senzor task** svakih 500 ms očitava temperaturu i preda je **komunikacijskom tasku** koji je formatira i pošalje. Povezuje ih **red čekanja** (queue).

```

QueueHandle_t xTempQueue;                                /* veza senzor → komunikacija */

void vSensorTask(void *pv) {                               /* PROIZVOĐAČ podatka */
    float temp;
    for (;;) {
        temp = read_temperature();                        /* očitaj ADC + pretvori u °C */
        xQueueSend(xTempQueue, &temp, 0);                /* stavi vrijednost u red */
        vTaskDelay(pdMS_TO_TICKS(500));                  /* spavaj 500 ms → Blocked */
    }
}

void vCommTask(void *pv) {                                 /* POTROŠAČ podatka */
    float temp; char buf[32];
    for (;;) {
        /* blokiraj dok ne stigne vrijednost – 0% CPU dok čekamo */
        if (xQueueReceive(xTempQueue, &temp, portMAX_DELAY) == pdTRUE) {
            int n = snprintf(buf, sizeof(buf), "T=%.1f C\r\n", temp);
            uart_send(buf, n);                             /* sporo slanje, nije hitno */
        }
    }
}

void start(void) {
    xTempQueue = xQueueCreate(4, sizeof(float));           /* do 4 vrijednosti */
    xTaskCreate(vSensorTask, "SENS", 256, NULL, 3, NULL);   /* viši prioritet */
    xTaskCreate(vCommTask, "COMM", 384, NULL, 2, NULL);     /* niži prioritet */
    vTaskStartScheduler();
}

```

Isječak 2. Cjelovit mali primjer dva taska + red čekanja. Isti obrazac (proizvođač–potrošač) koristi i ovaj projekt.

5.5 Kako znamo je li stog dovoljno velik — izračun

Pri kreiranju taska zadajemo veličinu stoga (3. argument `xTaskCreate`) u **riječima**. Na Cortex-M jedna riječ = 4 bajta, pa je 256 riječi = 1024 bajta. Procjena ide u tri koraka: analitička gornja granica → mjerenje → rezerva.

Korak A — analitička procjena za `vCommTask`

Zbrojimo sve što task stavlja na stog u svom najdubljem putu izvođenja:

Doprinos stogu	Procjena	Obrazloženje
Lokalne varijable	≈ 40 B	char buf[32] = 32 B, float temp = 4 B, int n = 4 B
Poziv <code>snprintf</code> s %f	≈ 150–200 B	formatiranje floata je «stack-žedno» (interni baferi)
Okviri ugnježđenih poziva	≈ 80 B	spremanje registara kroz nekoliko razina poziva
Iznimka/prekid usred taska	≈ 100 B	Cortex-M sprema 8 registara (32 B) + lijeni FPU (~68 B)
Međuzbroj (vrh potrošnje)	≈ 400 B	= 100 riječi
Sigurnosna rezerva (~50%)	≈ 200 B	za neviđene grane/dublje pozive
Preporuka	≈ 600 B → 256 r	zaokruženo naviše na 256 riječi (1024 B)

Tablica 7. Procjena stoga za `vCommTask`. Zaključak: 256 riječi je sigurno; 384 (kao u kodu) daje udobnu rezervu.

Za **`vSensorTask`** je račun manji (nema `snprintf`): par lokalnih varijabli + rezerva → 128–256 riječi je dovoljno. Pravilo: task koji zove `printf`/`snprintf`, plutajući zarez ili rekurziju treba znatno više stoga od taska koji samo prebacuje cijele brojeve.

Korak B — mjerenje u radu (*high-water mark*)

Procjena nije dokaz. FreeRTOS nudi funkciju koja vraća **najmanju količinu ikad slobodnog stoga** za task — pustimo sustav da odradi sve rubne slučajeve, pa očitamo koliko je rezerve stvarno ostalo:

```
/* unutar taska, povremeno ili na kraju testa: */
UBaseType_t slobodno = uxTaskGetStackHighWaterMark(NULL);
/* npr. vrati 180 → ostalo je 180 RIJEČI nikad iskorištenog stoga.
   Ako vrati 5 → stog je gotovo pun, hitno ga povećaj! */
```

Isječak 3. Mjerenje «vrha» potrošnje stoga. Cilj: ostaviti barem 25–50% rezerve iznad izmjerenog vrha.

Korak C — zaštita (ako ipak pogriješimo)

Uključimo automatsku provjeru prelijevanja. Ako task premaši stog, RTOS pozove naš «hook» u kojem zaustavimo sustav — lako uočljivo u debuggeru, umjesto nasumičnog rušenja kasnije:

```
/* FreeRTOSConfig.h */
#define configCHECK_FOR_STACK_OVERFLOW 2 /* provjeri uzorak na dnu stoga */

/* aplikacija: */
void vApplicationStackOverflowHook(TaskHandle_t t, char *name) {
    /* ovdje smo => task 'name' je prešao svoj stog */
    taskDISABLE_INTERRUPTS();
    for (;;) { }
}
```

Isječak 4. Mreža sigurnosti za stog. Bolje stati ovdje nego se srušiti «negdje drugdje» pola sekunde kasnije.

5.6 Kako odabrati prioritet

Prioritet se bira prema **hitnosti na vrijeme** (a ne prema važnosti posla!). Pitanje glasi: «što se dogodi ako ovaj task zakasni nekoliko milisekundi?»

- **Visok prioritet** → poslovi kratki i vremenski osjetljivi (reagiraj na događaj, uzmi svjež uzorak). U primjeru: vSensorTask (3) je viši jer se budi po tajmeru i mora uzeti uzorak s malim «podrhtavanjem» (jitterom).
- **Nizak prioritet** → poslovi dugi i tolerantni na kašnjenje. U primjeru: vCommTask (2) je niži jer je slanje sporo i nije strašno ako počne par ms kasnije.
- **Kratki na vrhu, dugi na dnu** → spriječi da spor task blokira hitan. Visokoprioritetni taskovi MORAJU često blokirati (vTaskDelay/queue), inače izgladnjuju niže.

Česta zamka: prioritetna inverzija

Ako niskoprioritetni task drži resurs (npr. zaključan mutex) koji treba visokoprioritetni, visoki čeka niskog — kao da su prioriteti zamijenjeni. Rješenje je **mutex s nasljeđivanjem prioriteta** (FreeRTOS ga ima): dok drži mutex, niski task privremeno dobije visoki prioritet da brže oslobodi resurs.

5.7 Queue, semafor ili mutex — kada koji

Tri su osnovna mehanizma za sinkronizaciju i razmjenu. Lako ih je zamijeniti, pa evo jasnog kriterija:

Mehanizam	Što radi	Kada ga koristiti
Queue (red)	Prenosi KOPIJU podatka + sinkronizira + razdvaja brzine.	Kad jedan task/ISR proizvodi PODATAK koji drugi treba (npr. izmjerena temperatura).
Binarni semafor	Samo signalizira DOGAĐAJ (0/1), bez podatka.	Kad treba samo «probuditi» task da nešto napravi (npr. «DMA gotov, buffer spreman»).
Brojeći semafor	Broji događaje/slobodne resurse.	Kad ima više jedinica resursa ili se događaji mogu nakupiti (npr. 5 slobodnih mjesta).
Mutex	Štiti zajednički resurs (uz nasljeđivanje prioriteta).	Kad više taskova dijeli istu strukturu/periferiju koju ne smiju mijenjati istovremeno.

Tablica 8. Izbor mehanizma. Pravilo palca: trebaš li prenijeti PODATAK → queue; samo SIGNAL → semafor; ČUVATI resurs → mutex.

Zašto u našem primjeru queue, a ne semafor

Senzor tasku treba prenijeti **stvarnu vrijednost** temperature komunikacijskom tasku. Queue to radi izravno: kopira float, usput budi potrošača i razdvaja brzine (proizvođač 2 Hz, potrošač svojim tempom). Binarni semafor sam po sebi **ne nosi podatak** — morali bismo dodati dijeljenu globalnu varijablu i zasebno je štiti, što je više koda i podložnije greškama (utrka, torn read).

Kada bi semafor bio bolji

Da senzor task samo treba reći «ima novih podataka u već dogovorenom bufferu» (bez prenošenja vrijednosti), binarni semafor bio bi lakši i brži. Tipičan slučaj je ISR koji javlja tasku «periferija je gotova» — nema vrijednosti za prenijeti, samo signal. Mutex pak ne služi za prenošenje nego isključivo za zaštitu: npr. dvije niti koje pišu u istu listu.

Most prema projektu

U ovom projektu DMA prekid šalje u queue broj 0 ili 1 (koja je polovica buffera gotova). Budući da nosi **podatak** (0/1), queue je ispravan izbor; da nije bilo razlike između polovica, dostajao bi binarni semafor. Na ESP32 strani lista WebSocket klijenata zaštićena je **mutexom** jer joj pristupa više niti. Oba primjera iz prakse vidjet ćete u poglavljima 7 i 11.

6. STM32 lanac akvizicije: TIM8 → ADC → DMA → ISR

Prije bilo kakve matematike treba **pretvoriti zvuk u niz brojeva** pouzdanom, ravnomjernom brzinom. Ovaj lanac to radi gotovo bez sudjelovanja procesora — hardver sam «puni» memoriju, a procesor se javlja tek kad je blok podataka spreman. To je ključ za rad u stvarnom vremenu.

6.1 TIM8 — generator takta uzorkovanja

Tajmer TIM8 postavljen je da se preljeva (overflow) točno 64 000 puta u sekundi. Pri svakom preljevu šalje interni okidač (TRGO) prema ADC-u. Time je **brzina uzorkovanja potpuno hardverska i jednolika** — ne ovisi o tome koliko je procesor zauzet.

```
TIM_InitStruct.Prescaler = 0;
TIM_InitStruct.AutoReload = 2655; /* ARR */
LL_TIM_SetTriggerOutput(TIM8, LL_TIM_TRGO_UPDATE); /* TRGO na svaki update */

/* Frekvencija = f_tim / (ARR + 1) = 170 MHz / 2656 = 64 006 Hz ≈ 64 kHz
   Perioda = 1 / 64000 = 15.625 μs po uzorku */
```

Isječak 5. Konfiguracija TIM8 (timer_driver.c / main.c). ARR = 2655 daje ≈ 64 kHz.

6.2 ADC — jedan pretvarač, četiri kanala redom

Sustav koristi **jedan** ADC (ADC1) koji u «scan» načinu pretvara četiri kanala jedan za drugim pri svakom okidaču tajmera. Budući da kanali nisu uzorkovani istovremeno nego redom, svaki sljedeći kasni za prethodnim za vrijeme jedne konverzije. To unosi sustavnu pogrešku u TDOA koju kod kasnije ispravlja (vidi 9.6).

```
/* ADC takt = PCLK / 4 = 170 MHz / 4 = 42.5 MHz → perioda 23.53 ns
   Po kanalu: 2.5 (uzorkovanje) + 12.5 (konverzija) = 15 ciklusa
               = 15 × 23.53 ns = 352.9 ns */
#define CH_DELAY_S 352.9e-9f /* M1=+0, M2=+1×, M3=+2×, M4=+3× */
```

Isječak 6. Kanalni offset zbog sekvencijalne konverzije (audio_common.h).

6.3 DMA — hardver puni memoriju umjesto procesora

DMA (Direct Memory Access) premješta svaki rezultat ADC-a izravno u RAM **bez prekidanja procesora za svaki uzorak**. Konfiguriran je kao **kružni**: kad napuni kraj buffera, automatski se vrati na početak. Buffer je dvostruke veličine i koristi se kao **dvostruki spremnik**:

- Dok DMA puni **drugu** polovicu buffera, procesor smije čitati **prvu** — i obrnuto. Nikad ne pišu i ne čitaju isti dio.
- DMA javlja dva prekida: **HT** (Half Transfer) i **TC** (Transfer Complete).

```
#define SAMPLES_PER_CHANNEL 1024
#define NUM_CH 4
#define HALF_BUFFER (NUM_CH * SAMPLES_PER_CHANNEL) /* 4096 uzoraka */
#define FULL_BUFFER (HALF_BUFFER * 2) /* 8192 uzoraka */
uint16_t adc_buffer[FULL_BUFFER]; /* 8192 × 2 B = 16 KB u RAM-u */

/* INTERLEAVED raspored (kanali isprepleteni):
   adc_buffer[s*4 + 0]=M1 [s*4+1]=M2 [s*4+2]=M3 [s*4+3]=M4 */
```

Isječak 7. Buffer i interleaveani raspored (adc_driver.c, audio_common.h).

Jedna «polovica» = 1024 vremenska trenutka × 4 kanala. Pri 64 kHz, 1024 uzorka po kanalu traje $1024 \div 64000$ = **16 ms**. Dakle svakih 16 ms stigne jedan HT ili TC prekid — to je «otkucaj srca» cijelog sustava.

6.4 ISR — kratak prekid koji samo javlja «gotovo»

Prekidna rutina ne radi nikakvu obradu — samo pošalje broj (0 za HT, 1 za TC) u red čekanja i, ako je time probudila task višeg prioriteta, zatraži preraspodjelu procesora. Sva «teška» obrada događa se izvan prekida, u tasku.

```
void DMA1_Channel1_IRQHandler(void) {
    BaseType_t woken = pdFALSE; uint32_t msg;
    if (LL_DMA_IsActiveFlag_HT1(DMA1)) { /* prva polovica gotova */
        LL_DMA_ClearFlag_HT1(DMA1); msg = 0u;
        xQueueSendFromISR(queueDmaEventHandle, &msg, &woken);
    }
    if (LL_DMA_IsActiveFlag_TC1(DMA1)) { /* druga polovica gotova */
        LL_DMA_ClearFlag_TC1(DMA1); msg = 1u;
        xQueueSendFromISR(queueDmaEventHandle, &msg, &woken);
    }
    portYIELD_FROM_ISR(woken); /* probudi ACQ_Task ako treba */
}
```

Isječak 8. DMA prekidna rutina (stm32g4xx_it.c). Pravilo: u ISR-u radi minimum, ostalo prepusti tasku.

7. FreeRTOS u ovom projektu: tri taska i protok podataka

Sada primjenjujemo načela iz poglavlja 5. Sustav koristi tri taska povezana s tri reda čekanja po obrascu **proizvođač–potrošač**: svaki task uzima posao iz ulaznog reda, obradi ga i preda u sljedeći.

Task	Prioritet	Stog	Uloga
ACQ_Task	Realtime (najviši)	256 riječi	Kopira svjež u polovicu buffera u snapshot
FFT_Task	High	1024 riječi	GCC-PHAT + 3D geometrija (sva matematika)
UART_Task	Low (najniži)	256 riječi	Šalje rezultat (i sirove uzorke) na ESP32

Tablica 9. Tri taska, prioriteta i stogovi (task_manager.c). Usporedite logiku prioriteta s odjeljkom 5.6.

Logika prioriteta (po pravilu iz 5.6 — hitnost na vrijeme): **akvizicija je najhitnija** jer propušten svjež podatak iz DMA-a nestaje zauvijek. Obrada (FFT) je važna ali smije malo pričekati. Slanje na UART je najmanje hitno (prikaz toleriran je da kasni desetke ms), pa ima najniži prioritet.

7.1 Tri reda čekanja

```
queueDmaEventHandle : DMA ISR → ACQ_Task (dubina 8, uint32 događaj HT/TC)
queueSnapshotHandle : ACQ_Task → FFT_Task (dubina 2, indeks buffera uint8)
queueResultHandle   : FFT_Task → UART_Task (dubina 4, loc3d_result_t)
```

Isječak 9. Tri reda čekanja (task_manager.h/c). Svaki nosi PODATAK → zato queue, ne semafor (vidi 5.7).

7.2 ACQ_Task — siguran prijenos svježe polovice

ACQ_Task čeka događaj iz DMA prekida (0 = HT, 1 = TC), tu polovicu kopira u jedan od **tri** «snapshot» buffera i indeks pošalje dalje. Tri buffera i kopiranje sprječavaju da DMA (koji nikad ne staje) prepíše podatke dok ih FFT_Task još obrađuje.

```
static void StartACQTask(void const *argument) {
    uint8_t write_idx = 0;
    for (;;) {
        osEvent evt = osMessageGet(queueDmaEventHandle, osWaitForever);
        if (evt.status != osEventMessage) continue;
        /* ako FFT zaostaje i red je pun – preskoči, ne prepisuj buffer
           koji se možda još čita (sprječava «torn read») */
        if (uxQueueSpacesAvailable(queueSnapshotHandle) == 0u) continue;
        const uint16_t *src = (evt.value.v == 0u) ? &adc_buffer[0]
                                                : &adc_buffer[HALF_BUFFER];
        memcpy(acq_snapshot[write_idx], src, HALF_BUFFER*sizeof(uint16_t));
        xQueueSend(queueSnapshotHandle, &write_idx, 0u);
        write_idx = (write_idx + 1u) % ACQ_NUM_BUFFERS; /* 0→1→2→0 */
    }
}
```

Isječak 10. ACQ_Task (task_manager.c). Trostruki spremnik + provjera praznine reda = nema oštećenih podataka.

Pojam: torn read (poderano čitanje)

Ako jedan task čita niz dok ga drugi (ili DMA) istovremeno mijenja, dobije mješavinu starih i novih vrijednosti — besmislen podatak. Rješenje: kopiraj u zaseban buffer i nikad ne prepisuj onaj koji je još «u optičaju».

7.3 FFT_Task — klizni prozor i sva matematika

FFT_Task nadovezuje svjež u polovicu na prethodnu, tvoreći **klizni prozor** od $2 \times 1024 = 2048$ trenutaka. Razlog: pljesak može pasti na granicu dviju polovica; s dvostrukim prozorom uvijek imamo cijeli događaj na jednom

mjestu.

```
static void StartFFTask(void const *argument) {
    loc3d_result_t result; uint8_t read_idx;
    for (;;) {
        if (xQueueReceive(queueSnapshotHandle, &read_idx, portMAX_DELAY) != pdTRUE)
            continue;
        memcpy(&sliding_buf[0], &sliding_buf[HALF_BUFFER], HALF_BUFFER*2);
        memcpy(&sliding_buf[HALF_BUFFER], acq_snapshot[read_idx], HALF_BUFFER*2);
        if (!capture_ready) {
            if (LOC3D_Process(sliding_buf, &result)) { /* ← sva obrada ovdje */
                capture_ready = 1;
                xQueueSend(queueResultHandle, &result, 0);
            }
        }
    }
}
```

Isječak 11. FFT_Task (task_manager.c). LOC3D_Process je srce DSP-a — razrađeno u poglavlju 9.

7.4 UART_Task i mehanizam «capture_ready»

UART_Task šalje rezultat na ESP32, a opcionalno i sirove uzorke (8 KB) za analizu. Slanje 8 KB na 115200 bps traje ~0.7 s — vječnost za sustav koji okida svakih 16 ms. Zastavica capture_ready «zamrzne» obradu dok traje slanje, da se sirovi buffer ne prepíše usred prijenosa. To je jednostavan ručni «handshake» između dva taska.

```
static void StartUARTTask(void const *argument) {
    loc3d_result_t r;
    for (;;) {
        if (xQueueReceive(queueResultHandle, &r, portMAX_DELAY) == pdTRUE) {
            UART_SendAngle3DPacket(r.az_tenth, r.el_tenth, r.strength);
            if (capture_ready) {
                UART_SendRawCapture(dbg_raw_ch0, dbg_raw_ch1,
                                    dbg_raw_ch2, dbg_raw_ch3);
                capture_ready = 0; /* obrada se nastavlja */
            }
        }
    }
}
```

Isječak 12. UART_Task (task_manager.c).

8. Procjena veličine stoga u ovom projektu

Primijenimo postupak iz odjeljka 5.5 na stvarne taskove. Ključna projektna odluka je da **veliki nizovi NE žive na stogu** — deklarirani su kao static (u BSS segmentu), pa stog ostaje malen.

```
/* Veliki radni nizovi su 'static' → u BSS-u, NE na stogu FFT_Task-a: */
static float s_ch0[1024], s_ch1[1024], s_ch2[1024], s_ch3[1024]; /* 16 KB */
static float s_corr[1024]; /* 4 KB */
/* Unutar GCC_PHAT — također 'static': */
static float fft_x[1024], fft_y[1024], fft_c[1024]; /* 12 KB */
```

Isječak 13. Veliki nizovi su 'static' (sound_loc_3d.c, gcc_phat.c) — zato FFT_Task treba samo ~4 KB stoga.

Da su ti nizovi bili obične lokalne varijable, jedan poziv LOC3D_Process tražio bi preko 30 KB stoga — višestruko više od dodijeljenog. Ovako na stogu ostaju samo skalari (pokazivači, brojači, međurezultati float).

Slijedom koraka A–C iz 5.5: za FFT_Task analitička procjena (skalari + interni okviri CMSIS-DSP-a + FPU/prekid rezerva) staje u nekoliko stotina bajta; uz velikodušnu rezervu dodijeljeno je 1024 riječi (4 KB). ACQ i UART task rade samo s memcpy/UART pozivima → 256 riječi svaki. Vrijednosti se potvrđuju mjerenjem (high-water mark) i štite hookom za prelijevanje.

Zašto je configENABLE_FPU = 1 obavezan ovdje

FFT_Task intenzivno koristi plutajući zarez (asinf, atan2f, množenja matrice). Ako raspoređivač prekine task usred float-operacije, a druga nit dotakne FPU, bez čuvanja FPU konteksta vrijednosti se nepovratno pokvare. Uz FPU=1 FreeRTOS koristi «lijeno» spremanje FPU registara — čuva ih samo kad task stvarno koristi FPU.

8.1 Bilanca FreeRTOS heap-a

Osim stogova, FreeRTOS ima zajednički **heap** iz kojeg alocira stogove taskova, redove čekanja i druge objekte. Postavljen je na 24 KB:

Objekt	Procjena
ACQ_Task stog (256 riječi)	1.0 KB
FFT_Task stog (1024 riječi)	4.0 KB
UART_Task stog (256 riječi)	1.0 KB
3 reda čekanja + RTOS overhead	≈ 0.6 KB
Ukupno zauzeto	≈ 6.6 KB
configTOTAL_HEAP_SIZE	24 KB
Slobodno za budući rast	≈ 17 KB

Tablica 10. Bilanca heap-a (FreeRTOSConfig.h). STM32G474RE ima 128 KB RAM-a — 24 KB je ~19%.

Napomena: veliki nizovi (adc_buffer 16 KB, snapshoti 24 KB, klizni prozor 16 KB, DSP nizovi ~32 KB) **ne dolaze iz heap-a** — statički su globalni (BSS) i linker ih smješta zasebno. Heap služi isključivo RTOS objektima.

9. Detekcija i lokalizacija zvuka — korak po korak (GCC-PHAT)

Ovo je srž projekta — funkcija LOC3D_Process(), koju FFT_Task zove za svaku novu polovicu (svakih 16 ms). Oslanja se na FFT iz poglavlja 4. Svaki korak prati **brojčani primjer** za jedan zamišljeni pljesak.

Postavka primjera koji pratimo kroz cijelo poglavlje

Pljesak dolazi iz smjera azimut +30°, elevacija +20°, dovoljno glasan. Kroz korake vidimo kako iz sirovih uzoraka ispadnu upravo ti kutovi.

9.1 Korak 0 — Cooldown (sprječavanje višestrukih okidanja)

Pljesak odzvanja desetke ms i prelijeva se preko više prozora. Bez kočnice sustav bi za jedan pljesak ispalio 5–10 detekcija. Nakon svake uspješne detekcije slijedi «hlađenje» od 19 prozora (≈ 304 ms).

```
#define DETECTION_COOLDOWN_FRAMES 19 /* 19 × 16 ms ≈ 304 ms */
if (s_cooldown > 0) { s_cooldown--; return 0; } /* preskoči, jeftino */
```

Isječak 14. Cooldown (sound_loc_3d.c).

9.2 Korak 1 — Pronalazak energetske vrha

Algoritam pronade gdje **unutar** kliznog prozora leži najglasniji dio i ondje centrira analizu. Koristi klizeći prozor energije (16 uzoraka) koji se pomiče inkrementalno — oduzme uzorak koji «izlazi», doda onaj koji «ulazi» ($O(N)$ umjesto $O(N \times W)$).

```
int32_t v = (int32_t)s[ch] - 2048; /* 2048 = sredina 12-bitnog raspona */
win_e += (v * v) >> 6; /* >>6 sprječava preljev */
win_e -= (v_old*v_old) >> 6; /* klizni pomak: skinu stari rub */
win_e += (v_new*v_new) >> 6; /* dodaj novi rub */
if (win_e > best_e) { best_e = win_e; best_frame = f + PROBE/2; }
```

Isječak 15. *find_peak_offset (sound_loc_3d.c)*. Rezultat: *frame_offset* = početak 1024-prozora centriranog na pljesak.

Primjer 9.2

Ako je pljesak najglasniji oko 1200-tog trenutka (od 0–2047), algoritam vrati *frame_offset* $\approx 1200 - 512 = 688$ → analizira uzorke [688 ... 1711], s događajem u sredini.

9.3 Korak 2 — Deinterleave, uklanjanje DC-a i Hann prozor

Iz interleavednog buffera izdvajamo 4 kanala i pripremamo ih za FFT (usporedi 4.10):

- **Uklanjanje DC-a.** Mikrofon «sjedi» na ~ 1.65 V (ADC ~ 2048). Za korelaciju nas zanima samo promjena oko sredine, pa oduzmemo prosjek kanala.
- **Hann prozor.** Glatko zvono na rubovima → 0 smanjuje spektralno curenje (objašnjeno u 4.10).

```
float w = s_hann[s]; /* unaprijed izračunato zvono */
ch0[s] = ((float)r0 - dc0) * w; /* (uzorak - DC) × prozor */
```

Isječak 16. *GCC_ExtractChannels (gcc_phat.c)*.

Primjer 9.3

Uzorak M1 = 2100, dc0 = 2048, Hann w = 0.95 → $(2100 - 2048) \times 0.95 = 49.4$. Tako «očišćen» signal ide u FFT.

9.4 Korak 3 — RMS prag (je li bilo dovoljno glasno)

RMS (Root Mean Square) je efektivna amplituda — mjera glasnoće. Pretih najglasniji kanal → tišina/šum → izlaz. Drugi (blaži) prag traži da ni najtiši kanal nije posve mrtav.

```
#define MIN_RMS_THRESHOLD 10.0f
#define MIN_RMS_PER_CHANNEL (MIN_RMS_THRESHOLD * 0.2f) /* = 2.0 */
if (rms_max < MIN_RMS_THRESHOLD) return 0; /* pretiho */
if (rms_min < MIN_RMS_PER_CHANNEL) return 0; /* jedan mik mrtav */
```

Isječak 17. *RMS prag (sound_loc_3d.c)*. $RMS = \sqrt{\text{prosjeck kvadrata uzoraka}}$.

Primjer 9.4

Pljesak rms = [42, 38, 35, 30]: rms_max=42>10 i rms_min=30>2 → prolazi. Tiha soba rms = [3,2,3,2]: rms_max=3<10 → izlaz.

9.5 Korak 4 — GCC-PHAT: kašnjenje između parova

Za svaki par (M1–M2, M1–M3, M1–M4) mjerimo za koliko uzoraka jedan signal kasni za drugim. Naivni pristup je **križna korelacija**: kliži jedan signal preko drugog i traži pomak najboljeg poklapanja. GCC-PHAT je njena izoštrana inačica koja radi preko FFT-a iz poglavlja 4.

Zašto PHAT, a ne obična korelacija

Obična korelacija daje širok, zaobljen vrh — teško je točno locirati maksimum, osobito uz jeku. **PHAT** (Phase Transform) u frekvencijskoj domeni **odbaci amplitudu i zadrži samo fazu**. Rezultat je vrlo oštar vrh točno na pravom kašnjenju — jer kašnjenje je linearni nagib faze po frekvenciji (sjetite se 4.6).

- 1 Izračunaj FFT oba signala: $X = \text{FFT}(\text{ref})$, $Y = \text{FFT}(\text{sig})$.
- 2 Križni spektar po frekvenciji: $C = \text{konj}(X) \cdot Y$.
- 3 PHAT normalizacija: podijeli svaki C s njegovim iznosom $|C|$ (ostaje samo faza).
- 4 Inverzni FFT od $C \rightarrow$ korelacija u vremenu; položaj vrha je kašnjenje.

```
arm_rfft_fast_f32(&s_rfft, ref, fft_x, 0); /* X = FFT(ref) */
arm_rfft_fast_f32(&s_rfft, sig, fft_y, 0); /* Y = FFT(sig) */
float cre = re1*re2 + im1*im2; /* Re{ konj(X)·Y } */
float cim = re1*im2 - im1*re2; /* Im{ konj(X)·Y } */
float mag = sqrtf(cre*cre + cim*cim);
float w = (mag > 1e-9f) ? 1.0f/(mag + 1e-9f) : 0.0f; /* PHAT: /|C| */
fft_c[iire] = cre*w; fft_c[iim] = cim*w;
arm_rfft_fast_f32(&s_rfft, fft_c, corr, 1); /* IFFT → korelacija */
```

Isječak 18. GCC_PHAT (gcc_phat.c). $\text{Konj}(X) \cdot Y$ daje vrh na pozitivnom kašnjenju kad sig kasni za ref.

Primjer 9.5 (predznak govori smjer)

Za par M1–M2: vrh na lagu +5 uzoraka znači da M2 kasni 5 uzoraka za M1 $\rightarrow$ val je do M1 stigao prije $\rightarrow$ izvor je bliži M1-strani. Predznak i iznos sva tri kašnjenja zajedno jednoznačno kodiraju smjer.

9.6 Korak 5 — Točan vrh i korekcija ADC offseta

Korelacija je uzorkovana u cijelim uzorcima, ali pravo kašnjenje rijetko pada točno na uzorak. **Parabolička interpolacija** kroz vrh i dva susjeda daje pod-uzorčano kašnjenje. Pretraga je ograničena na ± 20 uzoraka (granica iz poglavlja 3).

```
delta = 0.5f * (c0 - c2) / (c0 - 2*c1 + c2); /* tjeme parabole */
float frac = (float)max_idx + delta; /* npr. 5 + 0.3 = 5.3 */
return delay_samp * SAMPLE_PERIOD_S; /* uzorci → sekunde */
```

Isječak 19. GCC_FindTDOA (gcc_phat.c).

Korekcija sekvencijalnog ADC offseta: GCC-PHAT izmjeri **prividno** kašnjenje koje uključuje hardverski pomak (6.2). Stvarno **akustičko** kašnjenje = izmjereno + $(j-1) \times \text{CH_DELAY}$:

```
float tau12 = tau12_meas + 1.0f * CH_DELAY_S; /* M2 = 2. rank */
float tau13 = tau13_meas + 2.0f * CH_DELAY_S; /* M3 = 3. rank */
float tau14 = tau14_meas + 3.0f * CH_DELAY_S; /* M4 = 4. rank */
```

Isječak 20. Korekcija kanalnog offseta (sound_loc_3d.c).

Primjer 9.6 (naši brojevi)

Za $+30^\circ/+20^\circ$ prava akustička kašnjenja iz geometrije: $\tau_{12} \approx -274 \mu\text{s}$ (-17.5 uzoraka), $\tau_{13} \approx -137 \mu\text{s}$ (-8.8), $\tau_{14} \approx -198 \mu\text{s}$ (-12.7).

9.7 Korak 6 — Od kašnjenja do smjera (geometrija)

Tri kašnjenja i poznata geometrija jednoznačno određuju smjer. Veza je linearna: ako složimo baseline vektore u matricu $\mathbf{D}$ ($\text{reci} = \text{Mj} - \text{M1}$), vrijedi $\mathbf{D} \cdot \mathbf{s} = -\mathbf{c} \cdot \boldsymbol{\tau}$, gdje je $\mathbf{s}$ jedinični vektor prema izvoru, a $\mathbf{c}$ brzina zvuka.

```

/* jednom na startu: */ M_geom = c * inv(D), D = [M2-M1; M3-M1; M4-M1]
/* po detekciji:      */ u = M_geom *  $\tau$  (smjer propagacije)
ux = M_geom[0][0]* $\tau$ 12 + M_geom[0][1]* $\tau$ 13 + M_geom[0][2]* $\tau$ 14; /* itd. */
/* smjer PREMA izvoru = -u, normaliziran: */
sx=-ux; sy=-uy; sz=-uz; norm=sqrt(sx2+sy2+sz2); s/=norm;

```

Isječak 21. Geometrijsko rješenje (LOC3D_Init + LOC3D_Process, sound_loc_3d.c).

Zašto inverzija samo jednom

Matrica D ovisi samo o položajima mikrofona, koji se ne mijenjaju. Skupu inverziju napravimo jednom (LOC3D_Init) i spremimo; u vrućoj petlji ostaje samo jeftino množenje matrice s vektorom. Tipičan DSP obrazac: «pretproračunaj sve što je konstantno».

9.8 Korak 7 — Kutovi azimut i elevacija

```

float az_rad = atan2f(sy, sx); /* azimut: [-180°, +180°] */
float el_rad = asinf(sz); /* elevacija: [-90°, +90°] */
out->az_tenth = (int16_t)(az_rad * 1800.0f / PI); /* radijani → 0.1° */
out->el_tenth = (int16_t)(el_rad * 1800.0f / PI);

```

Isječak 22. Pretvorba vektora u kutove (sound_loc_3d.c).

Primjer 9.8 (zatvaranje kruga)

Geometrija vrati $s \approx (0.814, 0.470, 0.342)$. Tada $\text{atan2}(0.470, 0.814) = 30.0^\circ \rightarrow \text{az_tenth} = 300$, a $\text{asin}(0.342) = 20.0^\circ \rightarrow \text{el_tenth} = 200$. Točno smjer iz kojeg smo «pustili» pljesak.

9.9 Korak 8 — Jakost signala

```

float str_f = 20.0f * log10f(rms_max + 1.0f); /* dB-slično, log ljestvica */
if (str_f < 1.0f) str_f = 1.0f; if (str_f > 100.0f) str_f = 100.0f;
out->strength = (uint8_t)str_f; /* RMS 20→26, 100→40, 500→54 */

```

Isječak 23. Izračun jakosti (sound_loc_3d.c).

Na kraju se postavi cooldown i funkcija vrati 1 (valjana detekcija); FFT_Task rezultat šalje u red prema UART tasku.

10. Lokalizacija u vremenskoj domeni (3 i 4 mikrofona)

GCC-PHAT iz poglavlja 9 radi u **frekvencijskoj domeni** (FFT) i robustan je na šum i odjeke, ali je računski zahtjevan i osjetljiv na to da svi mikrofoni rade. Ovaj projekt sadrži i jednostavniju, brzu alternativu koja radi **izravno u vremenu, bez ijednog FFT-a: onset-threshold** metodu. Ona je posebno korisna za kratke, glasne tranzijente poput pljeska.

10.1 Ideja: tko prvi «čuje» pljesak

Pljesak je nagli skok amplitude. Umjesto da uspoređujemo cijele signale korelacijom, za svaki kanal samo tražimo **prvi uzorak u kojem signal naglo naraste** — trenutak dolaska (onset). Razlika tih trenutaka između mikrofona izravno je TDOA, izražena u cijelom broju uzoraka.

Onset u jednoj rečenici

«Okini» čim uzorak odstupa od mirne razine za više od praga: ako je $|\text{uzorak} - \text{DC}| > \text{THRESHOLD}$, zabilježi indeks tog uzorka kao trenutak dolaska zvuka na taj mikrofoni.

```
#define DC_LEVEL      2050      /* mirna razina (sredina 12-bitnog raspona) */
#define THRESHOLD_LEVEL 250      /* koliko mora «skočiti» da se broji kao zvuk */

static int32_t find_onset(const uint16_t *buf, uint32_t ch, int32_t *max_abs_out) {
    int32_t onset = -1, max_abs = 0;
    for (uint32_t s = 0; s < WIN_FRAMES; s++) {
        int32_t dev = (int32_t)buf[s*NUM_CH + ch] - DC_LEVEL; /* odmak od DC */
        if (dev < 0) dev = -dev; /* |odmak| */
        if (dev > max_abs) max_abs = dev; /* za jakost */
        if (onset < 0 && dev > THRESHOLD_LEVEL) onset = (int32_t)s; /* prvi skok */
    }
    *max_abs_out = max_abs;
    return onset; /* indeks prvog uzorka iznad praga, ili -1 ako tišina */
}
```

Isječak 24. Pronalazak oneta jednog kanala (*loc3d_3mic_time.c* / *sound_loc3d_4mic_time.c*).

DC razina je ovdje **fiksna konstanta** (2050), ne računa se po prozoru — to šteti jedan prolaz kroz podatke i čini metodu vrlo jeftinom. Prag 250 (od ~2048 sredine) znači da signal mora prijeći otprilike 6% punog raspona da bi se računao kao dolazak zvuka.

10.2 Od oneta do TDOA

Trenutke dolaska uvijek mjerimo **relativno na M1** (referentni mikrofoni), neovisno o tome koji je kanal fizički okinuo prvi. Razlika indeksa pomnožena s periodom uzorkovanja daje kašnjenje u sekundama:

```
int32_t d12 = onset[1] - onset[0]; /* M2 vs M1, u uzorcima */
int32_t d13 = onset[2] - onset[0]; /* M3 vs M1 */
int32_t d14 = onset[3] - onset[0]; /* M4 vs M1 (4-mic verz.) */

float tau12 = (float)d12 * SAMPLE_PERIOD_S + 1.0f*CH_DELAY_S; /* + ADC offset */
float tau13 = (float)d13 * SAMPLE_PERIOD_S + 2.0f*CH_DELAY_S;
float tau14 = (float)d14 * SAMPLE_PERIOD_S + 3.0f*CH_DELAY_S;
```

Isječak 25. TDOA iz razlike oneta + korekcija sekvencijalnog ADC offeta (*CH_DELAY_S* po ranku).

Dodatak $k \cdot \text{CH_DELAY_S}$ ispravlja činjenicu da ADC ne uzorkuje sve kanale istovremeno nego redom (M1, pa M2 nakon 870.6 ns, itd. — vidi poglavlje 6). Bez te korekcije unijeli bismo sustavnu pogrešku u kut.

Ako su oneti previše razmaknuti (više nego što je fizički moguće za zadanu geometriju), detekcija se odbacuje — to je zaštita od slučajnog šuma:

```
#define ONSET_MAX_SPREAD (TDOA_MAX_SAMPLES + 6)
if (d12 > ONSET_MAX_SPREAD || d12 < -ONSET_MAX_SPREAD || ...) return 0; /* odbaci */
```

Isječak 26. Provjera konzistentnosti: prevelik razmak onseta = nemoguće → odbaci okvir.

10.3 Geometrija: pravilan tetraedar

Da bismo iz TDOA dobili 3D smjer, trebamo poznavati položaje mikrofona. U ovoj verziji niz je **pravilan tetraedar** — sva četiri mikrofona međusobno su jednako udaljena (brid $a = 13$ cm). Tri mikrofona (M1, M2, M3) čine jednakostranični trokut u ravnini, a četvrti (M4) je vrh iznad njega.

Veličina	Formula	Vrijednost ($a = 13$ cm)
Stranica baze	a	13.00 cm
X baze (M2, M3)	$a \cdot \sqrt{3}/2$	11.2583 cm
Y baze ($\pm$)	$a/2$	± 6.50 cm
Centroid baze (X)	$a/\sqrt{3}$	7.5055 cm
Visina vrha M4	$a \cdot \sqrt{2/3}$	10.6145 cm

Tablica 11. Geometrija pravilnog tetraedra. M4 leži iznad centroida baze na visini $h = a \cdot \sqrt{2/3}$.

Zašto baš 10.61 cm visine

Za pravilan tetraedar visina vrha NIJE proizvoljna — određena je bridom: $h = a \cdot \sqrt{2/3} = 13 \cdot 0.8165 = 10.61$ cm. Tek na toj visini vrijedi $|M1-M4| = |M2-M4| = |M3-M4| = 13$ cm, tj. M4 je jednako udaljen od svih u bazi.

```
M1 = ( 0.0000, 0.0000, 0.0000) m RANK1 referentni
M2 = ( 0.1126, +0.0650, 0.0000) m RANK2 lijevo (+Y)
M3 = ( 0.1126, -0.0650, 0.0000) m RANK3 desno (-Y)
M4 = ( 0.0751, 0.0000, 0.1061) m RANK4 vrh tetraedra
```

Isječak 27. Koordinate mikrofona (audio_common.h). M4 = centroid baze (X,Y) + visina tetraedra (Z).

10.4 Tri mikrofona: ravnina + pretpostavka $z \geq 0$

S tri mikrofona u ravnini imamo samo **dvije** nezavisne TDOA jednadžbe (τ_{12} , τ_{13}). One određuju vodoravne komponente smjera (s_x , s_y), ali **ne** i okomitu (s_z). Verzija loc3d_3mic_time.c rješava 2×2 sustav za (s_x , s_y), a okomitu komponentu **rekonstruira uz pretpostavku da izvor dolazi odozgo ili iz ravnine** ($z \geq 0$):

```
/* [sx; sy] = -M_geom2 * [tau12; tau13] (M_geom2 = c*inv(D2), 2x2) */
float sxy2 = sx*sx + sy*sy;
float sz;
if (sxy2 > 1.0f) { /* vodoravni dio prejak → izvor na horizontu */
    float inv = 1.0f/sqrtf(sxy2);
    sx *= inv; sy *= inv; sz = 0.0f;
} else {
    sz = sqrtf(1.0f - sxy2); /* z >= 0: izvor iznad ravnine niza */
}
}
```

Isječak 28. 3-mic: (s_x, s_y) iz 2D sustava, s_z rekonstruiran iz uvjeta $|s|=1$ uz $z \geq 0$ (loc3d_3mic_time.c).

Ova pretpostavka je razumna kad znamo da izvor ne dolazi ispod niza (npr. niz stoji na stolu). Mana: sustav ne može razlikovati zvuk «odozgo» od zvuka «odozdo» — uvijek pretpostavlja gore.

10.5 Četiri mikrofona: puni 3D sustav (3×3)

S četvrtim mikrofonom (M4 izvan ravnine) dobivamo **treću** nezavisnu TDOA jednadžbu (τ_{14}). Sada možemo riješiti puni 3×3 sustav i dobiti **pravu** okomitu komponentu, uključujući njezin predznak — bez pretpostavke:

```

/* u = M_geom · tau ; M_geom = c · inv(D), D = [M2-M1; M3-M1; M4-M1] (3×3) */
float ux = M_geom[0][0]*tau12 + M_geom[0][1]*tau13 + M_geom[0][2]*tau14;
float uy = M_geom[1][0]*tau12 + M_geom[1][1]*tau13 + M_geom[1][2]*tau14;
float uz = M_geom[2][0]*tau12 + M_geom[2][1]*tau13 + M_geom[2][2]*tau14;
float sx = -ux, sy = -uy, sz = -uz; /* smjer PREMA izvoru = -smjer širenja */
/* normaliziraj na jedinični vektor ... */

```

Isječak 29. 4-mic: puni 3D smjer iz tri TDOA, $M_geom = c \cdot inv(D)$ (sound_loc3d_4mic_time.c).

U praksi je M4 (RANK4) često najslabiji/najšumniji kanal pa zna dati lažnu **negativnu** elevaciju («pljesneš iznad, detektira dolje»). Zato je dodana ista zaštita kao u 3-mic verziji: ako puni sustav vrati $sz < 0$, ne vjerujemo z-komponenti nego zadržimo (sx, sy) i rekonstruiramo $sz \geq 0$:

```

if (sz < 0.0f) { /* sumnjiva negativna elevacija (M4 šum) */
    float sxy2 = sx*sx + sy*sy;
    if (sxy2 > 1.0f) { float inv = 1.0f/sqrtf(sxy2); sx*=inv; sy*=inv; sz=0.0f; }
    else { sz = sqrtf(1.0f - sxy2); } /* zrcali u z >= 0 */
}

```

Isječak 30. 4-mic uz $z \geq 0$ zaštitu: M4 doprinosi vodoravnom rješenju, ali ne gura kut ispod horizonta.

10.6 Kutovi i rezolucija

Iz jediničnog vektora smjera kutovi se računaju isto kao u GCC-PHAT verziji: azimut = $\text{atan2}(sy, sx)$ (omotan u $[0^\circ, 360^\circ)$), elevacija = $\text{asin}(sz)$ (ovdje $[0^\circ, +90^\circ]$ zbog $z \geq 0$). Bitna razlika prema GCC-PHAT-u je **rezolucija**: onset daje samo **cijeli broj uzoraka**, dok GCC-PHAT paraboličkom interpolacijom postiže pod-uzorak.

Svojstvo	Time-domain (onset)	GCC-PHAT (FFT)
Računska cijena	vrlo niska (bez FFT)	viša (2 FFT + 1 IFFT po paru)
Rezolucija TDOA	1 uzorak (cijeli broj)	pod-uzorak (interpolacija)
Otpornost na šum/odjek	slabija	bolja (PHAT izoštravanje)
Osjetljivost na razine	veća (prag fiksna)	manja
Najbolje za	kratki glasni pljesak	općenite zvukove, reverberaciju)

Tablica 12. Usporedba dvaju pristupa. Onset je brz i jednostavan; GCC-PHAT je precizniji i robusniji.

Pri brzini uzorkovanja 192 kHz jedan uzorak odgovara putnoj razlici od $343 \text{ m/s} \div 192000 = 1.79 \text{ mm}$. To je gruba prostorna rezolucija — dovoljna za okvirni smjer pljeska, ali zrnata za elevaciju (gdje su razlike male). Izbor metode bira se pretprocesorskim zastavicama u `audio_common.h`:

```

#define USE_4MIC_TIME_LOC 1 /* 1 = 4-mic time-domain (ima prioritet) */
#define USE_3MIC_LOC 1 /* 1 = 3-mic (kad je gornji 0) */
#define USE_TIME_DOMAIN_LOC 1 /* 1 = onset (time), 0 = GCC-PHAT (kad 3-mic) */

```

Isječak 31. Odabir metode lokalizacije pri prevođenju (`audio_common.h`).

11. IMU senzor (MPU-6500/9250): I2C i orijentacija plohe

Da bismo na 3D vizualizaciji prikazali smjer zvuka u **stvarnom svjetskom okviru**, a ne samo relativno na ploču, moramo znati **kako je sama ploča nagnuta**. Tu ulogu ima IMU (Inertial Measurement Unit) — senzor MPU-6500/9250 na GY-521 modulu, spojen na STM32 preko **I2C** sabirnice.

11.1 Što IMU mjeri

- **Akcelerometar** — mjeri ubrzanje po tri osi. U mirovanju «vidi» samo gravitaciju (1 g prema dolje), pa iz njezina smjera možemo izračunati nagib (pitch i roll) — stabilno i bez drifta.
- **Žiroskop** — mjeri brzinu vrtnje po tri osi. Integracijom daje promjenu kuta, ali se pogreška s vremenom nakuplja (drift).
- **Magnetometar** (samo MPU-9250/9255) — mjeri Zemljino magnetsko polje, služi kao «kompas» za apsolutni smjer (yaw). MPU-6500 ga NEMA.

Zašto kombiniramo senzore (fuzija)

Akcelerometar je točan dugoročno ali «trza» na svaki pokret; žiroskop je gladak kratkoročno ali drifta. Spajanjem (fuzijom) uzimamo najbolje od oba: gladak kratkoročni odziv žiroskopa + dugoročnu stabilnost akcelerometra.

11.2 I2C sabirnica — dvije žice, više uređaja

I2C je serijska sabirnica s **dvije linije**: SCL (takt) i SDA (podaci). Svaki uređaj ima 7-bitnu adresu. STM32 je **master** (vodi komunikaciju), MPU je **slave**. U ovom projektu koristi se I2C1 na pinovima PB8 (SCL) i PB9 (SDA) pri 400 kHz (Fast-mode).

Signal	STM32 pin	GY-521 pin	Napomena
SCL (takt)	PB8	SCL	I2C1, AF4, open-drain + pull-up
SDA (podaci)	PB9	SDA	I2C1, AF4, open-drain + pull-up
Napajanje	3.3 V	VCC	modul ima regulator
Masa	GND	GND	—
Adresni odabir	GND	AD0	AD0=GND → adresa 0x68

Tablica 13. Spoj IMU-a na STM32 (i2c_driver.c). AD0 na GND daje I2C adresu 0x68 (na VCC bi bila 0x69).

Komunikacija s registrom uvijek ide u dva koraka: master kaže «želim registar broj R» (upis adrese registra), pa zatim čita ili piše podatke. Za čitanje se koristi «ponovljeni start» (repeated start) — bez puštanja sabirnice između upisa adrese i čitanja:

```
int I2C1_ReadReg(uint8_t dev, uint8_t reg, uint8_t *data, uint16_t len) {
    /* faza 1: pošalji broj registra (bez STOP-a → slijedi repeated start) */
    LL_I2C_HandleTransfer(I2C1, dev<<1, LL_I2C_ADDRSLAVE_7BIT, 1,
                          LL_I2C_MODE_SOFTEND, LL_I2C_GENERATE_START_WRITE);
    /* ... čekaj TXIS, pošalji reg, čekaj TC ... */
    /* faza 2: repeated start za čitanje, AUTOEND šalje STOP na kraju */
    LL_I2C_HandleTransfer(I2C1, dev<<1, LL_I2C_ADDRSLAVE_7BIT, len,
                          LL_I2C_MODE_AUTOEND, LL_I2C_GENERATE_START_READ);
    /* ... čitaj 'len' bajtova iz RXDR ... */
}
```

Isječak 32. Čitanje registra preko LL I2C funkcija (i2c_driver.c). Pisano LL bibliotekom, ne sirovim registrima.

11.3 WHO_AM_I — prepoznavanje točnog čipa

GY-521 moduli dolaze s različitim čipovima iste obitelji (6050, 6500, 9250, 9255) koji se izvana ne razlikuju. Točan čip se utvrđuje čitanjem registra **WHO_AM_I (0x75)** — svaki vraća jedinstveni identifikator:

WHO_AM_I	Čip	Magnetometar (kompas)
0x70	MPU-6500	NE — samo akcel + žiro
0x71	MPU-9250	DA (AK8963)
0x73	MPU-9255	DA (AK8963)
0x68	MPU-6050 (stari)	NE

Tablica 14. Identifikacija čipa po WHO_AM_I registru (mpu9250.c). U ovom projektu detektiran je MPU-6500 (0x70).

Driver pri pokretanju probava obje moguće adrese (0x68 i 0x69), pročita WHO_AM_I, i prema njemu odluči ima li čip magnetometar. Ako ga nema (kao kod MPU-6500), yaw nema apsolutnu referencu pa se NE šalje (postavi se 0).

```
mpu_chip_t MPU_Init(void) {
    if (I2C1_Ping(0x68) == 0) s_dev_addr = 0x68; /* probaj obje adrese */
    else if (I2C1_Ping(0x69) == 0) s_dev_addr = 0x69;
    else return MPU_CHIP_UNKNOWN; /* modul nije spojen */
    I2C1_ReadReg(s_dev_addr, MPU_REG_WHO_AM_I, &who, 1);
    switch (who) { /* prepoznaj čip */
        case 0x70: s_chip = MPU_CHIP_6500; break; /* bez kompasa */
        case 0x71: s_chip = MPU_CHIP_9250; break; /* + AK8963 */
        ...
    }
    /* probudi iz sleepa, ±2000°/s žiro, ±4g akcel, DLFP ~41 Hz ... */
}
```

Isječak 33. Detekcija čipa i inicijalizacija (mpu9250.c).

11.4 Mahony fuzija — od sirovih mjerenja do kutova

Sirova mjerenja (3 osi akcel + 3 osi žiro) treba spojiti u jednu konzistentnu orijentaciju. Koristimo **Mahony komplementarni filter**: lagan algoritam (prikladan za MCU) koji održava orijentaciju kao **kvaternion** i u svakom koraku je korigira.

- 1 Integriraj žiroskop → predviđa kako se orijentacija promijenila (gladak, ali drifta).
- 2 Iz akcelerometra procijeni smjer «dolje» (gravitacija) → daje apsolutnu referencu nagiba.
- 3 Razlika između predviđenog i izmjerenog «dolje» je pogreška; PI-regulator je vraća u žiroskopsku integraciju (ispravlja drift).
- 4 (Ako ima magnetometar) isto napravi za smjer sjevera → ispravlja yaw drift.
- 5 Kvaternion pretvori u Eulerove kutove: roll, pitch, yaw.

```
int MPU_Update(float dt_s, mpu_orientation_t *out) {
    read_accel_gyro(a, g); /* sirova mjerenja preko I2C */
    int mag_ok = (read_mag(m) == 0); /* magnetometar ako postoji */
    mahony_update(g, a, m, mag_ok, dt_s); /* fuzija → kvaternion (q0..q3) */
    /* kvaternion → Tait-Bryan kutovi (ZYX): */
    out->roll_deg = atan2f(...) * (180/PI);
    out->pitch_deg = asinf(...) * (180/PI);
    out->yaw_deg = atan2f(...) * (180/PI); /* 0..360, samo ako ima kompas */
    return 0;
}
```

Isječak 34. Jedan korak fuzije: čitaj senzore → Mahony → Eulerovi kutovi (mpu9250.c).

Pitch/roll rade, yaw ne — zašto

MPU-6500 nema magnetometar, pa yaw (rotacija oko vertikale) nema apsolutnu referencu i samo bi driftao. Zato sustav s ovim čipom šalje yaw = 0, a vizualizacija zakreće plohu samo po nagibu (pitch/roll) — stabilno iz gravitacije. Za pun heading treba čip s kompasom (9250/9255).

11.5 IMU task i slanje na ESP32

Čitanje senzora i fuzija žive u zasebnom **IMU_Task**-u (prioritet BelowNormal — ispod akvizicije i lokalizacije, da ih nikad ne ometa). Petlja radi fuziju na ~100 Hz, a orijentaciju šalje na ESP32 rjeđe (~20 Hz), novim tipom UART paketa **0x05**:

```
for (;;) {
    if (MPU_Update(0.01f, &ori) == 0) {           /* 100 Hz fuzija */
        if (++send_div >= 5) {                     /* svakih 5 → ~20 Hz slanje */
            send_div = 0;
            int16_t roll_t = ori.roll_deg*10, pitch_t = ori.pitch_deg*10;
            int16_t yaw_t = flags ? ori.yaw_deg*10 : 0; /* yaw 0 bez kompasa */
            osMutexWait(uartMutexHandle, osWaitForever); /* dijeli UART s 0x03 */
            UART_SendOrientationPacket(roll_t, pitch_t, yaw_t, flags);
            osMutexRelease(uartMutexHandle);
        }
    }
    osDelay(10);
}
```

Isječak 35. IMU_Task: fuzija na 100 Hz, slanje paketa 0x05 na ~20 Hz pod UART mutexom (task_manager.c).

UART4 sada dijele dva proizvođača: UART_Task (kutovi zvuka, 0x03) i IMU_Task (orijentacija, 0x05). Da se bajtovi dvaju paketa ne izmiješaju na žici, oba slanja štiti zajednički **mutex** (uartMutexHandle) — točno scenarij «zaštititi zajednički resurs» iz odjeljka 5.7.

11.6 Kompenzacija u vizualizaciji

STM32 šalje smjer zvuka u **lokalnom okviru ploče** (paket 0x03) i orijentaciju ploče (paket 0x05) **odvojeno**. Kompenzaciju radi ESP32 pri iscrtavanju: cijela scena (3 referentne ravnine, mikrofoni, zvučne kuglice) množi se rotacijskom matricom ploče, pa se prikazani smjer zakreće zajedno s fizičkim nagibom niza.

```
/* WebGL: model-matrica ploče iz orijentacije (ZYX kao Mahony) */
function boardM(){ return mm(rmZ(gYaw), mm(rmY(gPitch), rmX(gRoll))); }
/* svaki element scene množi se s boardM() prije iscrtavanja */
```

Isječak 36. ESP32 zakreće cijelu scenu prema orijentaciji (web_server.c). Bez magnetometra gYaw ostaje 0.

Zašto ESP32 radi kompenzaciju, a ne STM32

Razdvajanjem (zvuk u lokalnom okviru + orijentacija zasebno) ESP32 može prikazati i «sirovi» smjer i kompenzirani, lakše se debugira, a STM32 ostaje fokusiran na obradu zvuka. Cijena je minimalna — par matričnih množenja u pregledniku.

12. UART protokol između STM32 i ESP32

Ploče komuniciraju serijski (UART, 115200 bps, 8N1) jednostavnim **binarnim paketnim protokolom**. Svaki paket počinje s 0xAA 0xBB i završava s 0xCC 0xDD; između je bajt tipa i korisni teret. Start/end markeri omogućuju prijemu da se «uhvati» na početak paketa i nakon izgubljenog bajta.

Tip	Naziv	Duljina	Sadržaj
0x02	2D kut	8 B	AZ(2) + jakost(1)
0x03	3D kut (zvuk)	10 B	AZ(2) + EL(2) + jakost(1)
0x04	Sirovi capture	~8 KB	NCH + N + NCH×N×uint16 (big-endian)
0x05	Orijentacija (IMU)	12 B	ROLL(2) + PITCH(2) + YAW(2) + FLAGS(1)

Tablica 15. Tipovi paketa (uart_driver.c na obje strane). 0x03 = smjer zvuka, 0x05 = orijentacija ploče.

3D paket (tip 0x03), 10 bajtova – SMJER ZVUKA (lokalni okvir ploče):

AA	BB	03	AZ_H	AZ_L	EL_H	EL_L	STR	CC	DD
----	----	----	------	------	------	------	-----	----	----

Orijentacijski paket (tip 0x05), 12 bajtova – NAGIB PLOČE (IMU):

AA	BB	05	RL_H	RL_L	PT_H	PT_L	YW_H	YW_L	FLAG	CC	DD
----	----	----	------	------	------	------	------	------	------	----	----

ROLL, PITCH, YAW = int16 big-endian u desetinkama °; FLAGS bit0 = kompas valjan

Isječak 37. Raspored paketa 0x03 (zvuk) i 0x05 (orijentacija). Različit tip → ESP32 ih razlikuje.

13. ESP32: Wi-Fi, web-poslužitelj i 3D vizualizacija

ESP32 je «prezentacijski» dio. Pri pokretanju (app_main) redom inicijalizira: NVS, Wi-Fi (Access Point), HTTP poslužitelj, UART i procesor primljenih paketa.

```
void app_main(void) {
    nvs_flash_init();           /* trajna pohrana (Wi-Fi treba NVS) */
    wifi_manager_init();        /* podigni vlastitu Wi-Fi mrežu (AP) */
    web_server_init();          /* HTTP + WebSocket na portu 80 */
    uart_driver_init();          /* UART1: RX=GPI016, TX=GPI017, 115200 */
    sound_loc_processor_init(); /* pokreni rx_task (parser paketa) */
}
```

Isječak 38. Redoslijed inicijalizacije na ESP32 (main.c).

13.1 Wi-Fi Access Point

ESP32 ne spaja se na postojeću mrežu, nego **stvara vlastitu** — sustav radi samostalno, bez routera. Korisnik se spoji na tu mrežu i otvori adresu poslužitelja.

Parametar	Vrijednost
SSID (ime mreže)	SoundLocalization
Lozinka	soundloc123
IP poslužitelja	192.168.4.1
Maks. klijenata	4

Tablica 16. Wi-Fi AP postavke (wifi_manager.c). U pregledniku otvoriti <http://192.168.4.1>

13.2 Parser paketa — konačni automat (state machine)

UART podaci stižu kao **tok bajtova** bez jasnih granica. rx_task ih sklapa **konačnim automatom**: za svaki bajt prelazi u sljedeće stanje (SOF1 → SOF2 → tip → ... → EOF). Tek uz ispravan end-marker paket se smatra valjanim.

```
switch (state) {
    case S_SOF1: if (b==0xAA) state=S_SOF2; break; /* čekaj start */
    case S_SOF2: state=(b==0xBB)?S_TYPE:S_SOF1; break;
    case S_TYPE: pkt_type=b;
                  if (b==0x03) state=S_AZ_H; /* smjer zvuka */
                  else if (b==0x05) state=S_ORI_RL_H; /* orijentacija (IMU) */
                  else if (b==0x04) state=S_RAW_NCH; /* sirovi capture */
                  else state=S_SOF1; break;
    ...
    case S_EOF2: if (b==0xDD) {
                  if (pkt_type==0x05) /* orijentacija ploče */
                      web_server_send_orientation(roll, pitch, yaw, mag_ok);
                  else /* smjer zvuka */
                      web_server_send_data(az/10.0f, el/10.0f, str_val);
                  } state=S_SOF1; break;
}
```

Isječak 39. Parser razlikuje 0x03 (zvuk) i 0x05 (orijentacija) po tipu (sound_loc_processor.c).

Zašto automat, a ne «pročitaj 10 bajtova»

Ako se izgubi jedan bajt, naivno čitanje fiksne duljine ostalo bi zauvijek pomaknuto. Automat se uvijek iznova sinkronizira na 0xAA 0xBB, pa se nakon greške sam oporavi na sljedećem paketu.

13.3 WebSocket i mutex (veza s 5.7)

Za podatke koji stižu spontano (svaki pljesak) prikladniji je **WebSocket** od klasičnog HTTP-a — trajna veza preko koje poslužitelj **sam** gurne podatak čim ga ima. Za svaku detekciju ESP32 svim preglednicima pošalje JSON.

```
{ "type": "sound", "azimuth": 30.0, "polar": 20.0, "strength": 42 } /* smjer zvuka */
{ "type": "orient", "roll": 5.0, "pitch": -3.0, "yaw": 0.0, "mag": 0 } /* nagib ploče */
```

Isječak 40. JSON poruke preko WebSocket-a — tip «sound» (zvuk) i «orient» (orijentacija) (web_server.c).

Lista spojenih klijenata čuva se uz **mutex** (ws_fds_mutex) jer joj pristupaju dvije niti — ona koja prima nove veze i ona koja šalje podatke. To je upravo slučaj iz odjeljka 5.7 «mutex za zaštitu zajedničkog resursa»: bez zaključavanja lista bi se mogla oštetiti; mrtve veze automatski se uklanjaju.

13.4 3D vizualizacija (WebGL)

Web-stranica ugrađena je kao tekst u firmware ESP32 i poslužuje se na «/». Crta minimalnu 3D scenu (WebGL):

- Mikrofoni (M1–M4) iscrtani kao crvene kuglice na svojim položajima.
- Tri prozirne ravnine (XY, XZ, YZ) daju osjećaj prostora.
- Svaka detekcija dodaje **svjetleću kuglicu** na izračunatom smjeru, koja kroz 5 s polako izblijedi.
- Cijela scena (ravnine, mikrofoni, kuglice) **zakreće se prema orijentaciji ploče** iz IMU paketa 0x05 — tako prikaz prati fizički nagib niza (poglavlje 11.6).
- Mišem/dodirom scena se zakreće i zumira (orbit kamera).

Pretvorba kuta u 3D položaj kuglice

Iz azimuta (a) i elevacije (e): $x = 8 \cdot \cos(e) \cdot \cos(a)$, $y = 8 \cdot \cos(e) \cdot \sin(a)$, $z = 8 \cdot \sin(e)$. Tako kut iz STM32 izračuna postaje točka koju oko vidi; potom se cijeli vektor još množi rotacijskom matricom ploče (kompenzacija nagiba).

14. Cjeloviti primjer: od pljeska do točke na ekranu

Spojimo sve u jedan slijed za **jedan pljesak** iz smjera azimut $+30^\circ$, elevacija $+20^\circ$:

- 1 **t = 0 ms.** Val stiže na mikrofone u malo različitim trenucima.
- 2 **Kontinuirano.** TIM8 okida ADC svakih $15.6 \mu\text{s}$; ADC redom čita M1–M4; DMA puni adc_buffer bez procesora.
- 3 **Svakih 16 ms.** DMA javi HT/TC → ISR pošalje događaj u red → ACQ_Task kopira polovicu u snapshot i javi FFT_Task-u.
- 4 **FFT_Task.** Sastavi klizni prozor (2048), pozove LOC3D_Process: nađe energetski vrh, izdvoji kanale (DC + Hann), provjeri RMS prag.
- 5 **GCC-PHAT × 3.** Izmjeri $\tau_{12} \approx -274 \mu\text{s}$, $\tau_{13} \approx -137 \mu\text{s}$, $\tau_{14} \approx -198 \mu\text{s}$ (nakon korekcije offseta).
- 6 **Geometrija.** $u = M_{\text{geom}} \cdot \tau$, pa $s = -u/|u| \approx (0.814, 0.470, 0.342)$.
- 7 **Kutovi.** azimut = 30.0° , elevacija = 20.0° , jakost ≈ 42 .
- 8 **UART.** UART_Task pošalje [AA BB 03 01 2C 00 C8 2A CC DD] na ESP32.
- 9 **ESP32.** rx_task automatom sklopi paket, izračuna $30.0^\circ/20.0^\circ$, pošalje JSON preko WebSocket-a.
- 10 **Preglednik.** Doda svjetleću kuglicu na (x,y,z) za $30^\circ/20^\circ$; HUD ispiše «Azimuth: 30.0° | Polar: 20.0° | Strength: 42».
- 11 **Cooldown.** STM32 idućih ~ 304 ms ignorira nove detekcije.

$0x012C = 300 = 30.0^\circ$ (azimut), $0x00C8 = 200 = 20.0^\circ$ (elevacija), $0x2A = 42$ (jakost) — bajtovi u paketu iz koraka 8.

15. Pojmovnik za početnike

Pojam	Objašnjenje
ADC	Analog-to-Digital Converter — pretvara napon u broj (ovdje 12-bitni, 0–4095).
DMA	Direct Memory Access — sklop koji premješta podatke u/iz memorije bez procesora.
ISR	Interrupt Service Routine — kratka rutina koja se izvrši na hardverski prekid.
RTOS	Real-Time OS — sustav koji dijeli procesor na taskove s prioritetima.
Task	Neovisna nit izvođenja s vlastitim stogom; vrti se u petlji.
Scheduler	Raspoređivač — dio RTOS-a koji bira koji task trenutno radi.
Preemption	Oduzimanje procesora nižem tasku čim viši postane spreman.
Stack	Stog — memorija za lokalne varijable i povratne adrese jednog taska.
High-water mark	Najmanja ikad zabilježena slobodna rezerva stoga; mjeri stvarnu potrošnju.
Heap	Gomila — zajednička memorija iz koje RTOS alocira objekte.
Queue	Red čekanja — prenosi podatak među taskovima/ISR-om i sinkronizira ih.
Semafor	Signalizira događaj (binarni) ili broji resurse (brojeći); ne nosi podatak.
Mutex	Brava za zaštitu zajedničkog resursa, s nasljeđivanjem prioriteta.
DFT	Discrete Fourier Transform — rastav niza uzoraka na frekvencijske binove.
FFT	Fast Fourier Transform — brzi algoritam za DFT ($N \cdot \log_2 N$ umjesto N^2).
Bin	Frekvencijski pretinac DFT-a; razmak = f_s/N (kod nas 62.5 Hz).
Magnituda / faza	Iznos i kut kompleksnog $X[k]$: «koliko» i «s kojim pomakom» frekvencije.
Twiddle faktor	Kompleksni množitelj $e^{-j2\pi k/N}$ u leptiru FFT-a.
Hann prozor	Glatko «zvono» kojim se množi signal da se smanji spektralno curenje.
TDOA	Time Difference Of Arrival — razlika vremena dolaska zvuka na dva mikrofona.
Korelacija	Mjera sličnosti dvaju signala u ovisnosti o njihovom pomaku.
GCC-PHAT	Izoštrena križna korelacija (samo faza) za precizno mjerenje TDOA.
RMS	Root Mean Square — efektivna amplituda (mjera glasnoće) signala.
Azimet / elevacija	Vodoravni / okomiti kut smjera izvora.
Interleaved	Isprepleteni raspored uzoraka: M1 M2 M3 M4 M1 M2...
Big-endian	Zapis broja s višim bajtom prvim (npr. 300 = 0x01 0x2C).
WebSocket	Trajna dvosmjerna veza preglednik–poslužitelj za «push» podatke.
Access Point	Način rada u kojem ESP32 sam stvara Wi-Fi mrežu.
IMU	Inertial Measurement Unit — senzor pokreta (akcelerometar + žiroskop, ev. kompas).
Akcelerometar	Mjeri ubrzanje; u mirovanju «vidi» gravitaciju → daje nagib (pitch/roll).
Žiroskop	Mjeri brzinu vrtnje; integracijom daje kut, ali s vremenom drifta.
Magnetometar	«Kompas» — mjeri Zemljino polje za apsolutni yaw (samo MPU-9250/9255).
I2C	Serijska sabirnica s dvije linije (SCL takt, SDA podaci) i adresiranim uređajima.

Pojam	Objašnjenje
WHO_AM_I	Registar (0x75) koji vraća ID čipa — razlikuje MPU-6500/9250/9255.
Kvaternion	Četverodimenzionalni zapis orijentacije bez «gimbal lock» problema.
Mahony filter	Lagana fuzija akcel+žiro(+mag) u orijentaciju; ispravlja drift PI-regulatorom.
Pitch / roll / yaw	Nagib naprijed-nazad / lijevo-desno / rotacija oko vertikale.
Onset	Trenutak dolaska zvuka — prvi uzorak iznad praga amplitude (time-domain TDOA).
Tetraedar (pravilan)	Tijelo s 4 jednako udaljena vrha; ovdje raspored 4 mikrofona.

Kraj dokumenta (v3). Svi isječci koda i konstante preuzeti su izravno iz izvornog koda projekta (mape «Sound Localization» i «ESP32_Visualization»). Primjeri u poglavljima 4 i 5 namjerno koriste jednostavne, izmišljene vrijednosti radi učenja. Nova poglavlja 10 (vremenska domena) i 11 (IMU) opisuju funkcionalnosti dodane nakon druge verzije dokumenta.